

Unit 1: Collections

Collection types

- Array
- Dictionary
- Set
- Tuple

Arrays

Defining

```
[value1, value2, value3]
```

```
var names: [String] = ["Anne", "Gary", "Keith"]
```


Arrays

Defining

```
[value1, value2, value3]
```

```
var names = ["Anne", "Gary", "Keith"]
```

Arrays

Defining

```
var numbers = [1, -3, 50, 72, -95, 115]
```

Arrays

Defining

```
var numbers: [Int8] = [1, -3, 50, 72, -95, 115]
```


contains

```
let numbers = [4, 5, 6]
if numbers.contains(5) {
    print("There is a 5")
}
```

There is a 5

Array types

```
var myArray: [Int] = []
```

```
var myArray: Array<Int> = []
```

```
var myArray = [Int]()
```


Working with arrays

repeating

```
var myArray = [Int](repeating: 0, count: 100)
```

```
let count = myArray.count
```

```
if myArray.isEmpty { }
```

Working with arrays

Accessing or setting a specific item

```
var names = ["Anne", "Gary", "Keith"]  
let firstName = names[0]  
print(firstName)
```

Anne

```
names[1] = "Paul"  
print(names)
```

["Anne", "Paul", "Keith"]

Working with arrays

Appending

```
var names = ["Amy"]  
names.append("Joe")  
names += ["Keith", "Jane"]  
print(names)
```

```
["Amy", "Joe", "Keith", "Jane"]
```


Working with arrays

Inserting

```
var names = ["Amy", "Brad", "Chelsea", "Dan"]  
names.insert("Bob", at: 0)  
print(names)
```

```
["Bob", "Amy", "Brad", "Chelsea", "Dan"]
```

Working with arrays

Removing

```
var names = ["Amy", "Brad", "Chelsea", "Dan"]  
let chelsea = names.remove(at:2)  
let dan = names.removeLast()  
print(names)
```

```
["Amy", "Brad"]
```

```
names.removeAll()  
print(names)
```

```
[]
```

Working with arrays

```
var myNewArray = firstArray + secondArray“
```

```
let containerArray = [array1, array2] // [ [1,2,3], [4,5,6] ]
```

```
let firstArray = containerArray[0] // [1,2,3]
```

```
let firstElement = containerArray[0][0] // 1
```


Working with arrays

Arrays within arrays

```
let array1 = [1,2,3]
let array2 = [4,5,6]
let containerArray = [array1, array2]
let firstArray = containerArray[0]
let firstElement = containerArray[0][0]
print(containerArray)
print(firstArray)
print(firstElement)
```

```
[[1, 2, 3], [4, 5, 6]]
[1, 2, 3]
1
```

Other Array Methods

Other Array Methods

Method	Description
<code>sort()</code>	sorts array elements
<code>shuffle()</code>	changes the order of array elements
<code>forEach()</code>	calls a function for each element
<code>contains()</code>	searches for the element in an array
<code>swapAt()</code>	exchanges the position of array elements
<code>reverse()</code>	reverses the order of array elements

Dictionaries

[key1 : value1, key2: value2, key3: value3]

```
var scores = ["Richard": 500, "Luke": 400, "Cheryl": 800]
```

```
var myDictionary = [String: Int]()
```

```
var myDictionary = Dictionary<String, Int>()
```

```
var myDictionary: [String: Int] = [:]
```

Add/remove/modify a dictionary

Adding or modifying

```
var scores = ["Richard": 500, "Luke": 400, "Cheryl": 800]
```

```
scores["Oli"] = 399
```

```
let oldValue = scores.updateValue(100, forKey: "Richard")
```

Add/remove/modify a dictionary

Adding or modifying

```
var scores = ["Richard": 500, "Luke": 400, "Cheryl": 800]

scores["Oli"] = 399

if let oldValue = scores.updateValue(100, forKey: "Richard") {
    print("Richard's old value was \(oldValue)")
}
```

Richard's old value was 500

Add/remove/modify a dictionary

Removing

```
var scores = ["Richard": 100, "Luke": 400, "Cheryl": 800]
scores["Richard"] = nil
print(scores)

if let removedValue = scores.removeValue(forKey: "Luke") {
    print("Luke's score was \(removedValue) before he stopped playing")
}
print(scores)
```

["Cheryl": 800, "Luke": 400]

Luke's score was 400 before he stopped playing

["Cheryl": 800]

Accessing a dictionary

```
var scores = ["Richard": 500, "Luke": 400, "Cheryl": 800]

let players = Array(scores.keys) //["Richard", "Luke", "Cheryl"]
let points = Array(scores.values) //[500, 400, 800]

if let lukesScore = scores["Luke"] {
    print(lukesScore)
}

400

if let henrysScore = scores["Henry"] {
    print(henrysScore)
}
```

SET

- A set stores distinct values which are of the same type of collection with no ordering.
- You can use a set instead of an array when the **order of items isn't important**, or when you need to ensure that an **item only appears once**.

Hash Values for Set Types:

- The type of set values must be hashable in order to be stored in a set. All of Swift's basic types such as String, Int, Double and Bool are hashable by default.

SET

SET TYPE SYNTAX :

- The syntax of set in Swift is written as Set<Element>, where Element is the type of elements that is allowed to store:

```
var characters = Set<Character>()
characters.insert("c")
characters.count()    // Prints - 1
characters = []
characters.count()    // Prints - 0
```

A set type cannot be inferred from an array literal, so in such cases the type Set must be explicitly declared

SET

- A set type cannot be inferred from an array literal, so in such cases the type Set must be explicitly declared. Although, because of Swift's type inference, you do not have to write the type of the set's elements if you are initializing it with an array literal.

Creating and Initializing an Empty Set:

- You can create an empty set of a certain type using initializer syntax:

```
var letters = Set<Character>()  
print("letters is of type Set<Character> with \(letters.count) items.")
```

```
// Prints "letters is of type Set<Character> with 0 items."
```


SET

Alternatively, if the context already provides type information, such as a function argument or an already typed variable or constant, you can create an empty set with an empty array literal:

```
letters.insert("a")
```

```
// letters now contains 1 value of type Character
```

```
letters = []
```

```
// letters is now an empty set, but is still of type Set<Character>
```

Creating a Set with an Array Literal

- You can also initialize a set with an array literal, as a shorthand way to write one or more values as a set collection.
- The example below creates a set called favoriteGenres to store String values:

```
var favoriteGenres: Set<String> = ["Rock", "Classical", "Hip hop"]  
// favoriteGenres has been initialized with three initial items
```

- The initialization of favoriteGenres could have been written in a shorter form instead:

```
var favoriteGenres: Set = ["Rock", "Classical", "Hip hop"]
```

Accessing and Modifying a Set

- To find out the number of items in a set, check its read-only **count** property:

```
print("I have \(favoriteGenres.count) favorite music genres.")  
// Prints "I have 3 favorite music genres."
```

- Use the Boolean **isEmpty** property as a shortcut for checking whether the count property is equal to 0:

```
if favoriteGenres.isEmpty {  
    print("As far as music goes, I'm not picky.")  
} else {  
    print("I have particular music preferences.")  
}  
// Prints "I have particular music preferences."
```


Accessing and Modifying a Set

- You can add a new item into a set by calling the set's `insert(_:)` method:

```
favoriteGenres.insert("Jazz")  
// favoriteGenres now contains 4 items
```

- You can remove an item from a set by calling the set's `remove(_:)` method, which removes the item if it's a member of the set, and returns the removed value, or returns nil if the set didn't contain it.
- Alternatively, all items in a set can be removed with its `removeAll()` method.
- To check whether a set contains a particular item, use the `contains(_:)` method.

Iterating Over a Set

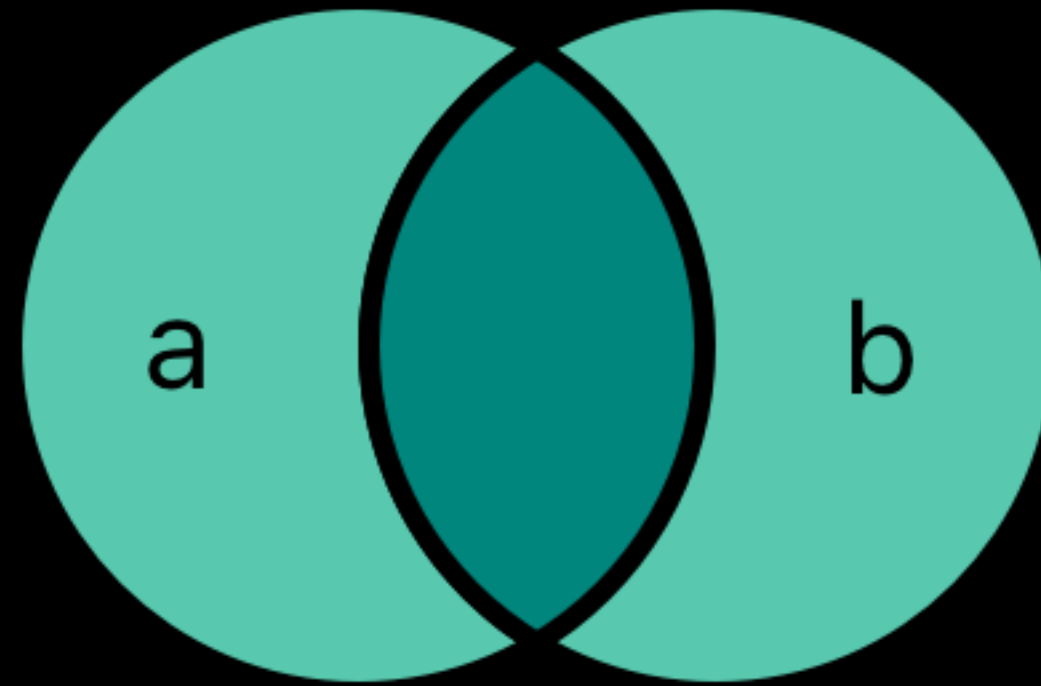
- You can iterate over the values in a set with a **for-in** loop.

```
for genre in favoriteGenres {  
    print("\(genre)")  
}  
// Classical  
// Jazz  
// Hip hop
```

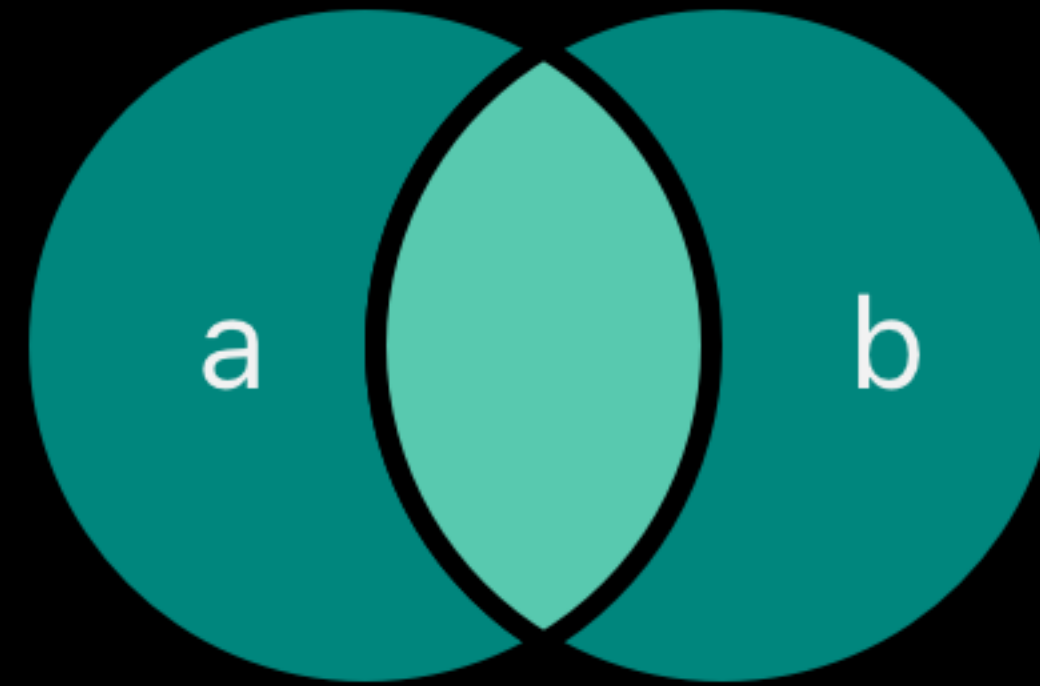
- Swift's Set type doesn't have a defined ordering. To iterate over the values of a set in a specific order, use the **sorted()** method, which returns the set's elements as an array sorted using the < operator.

Fundamental Set Operations

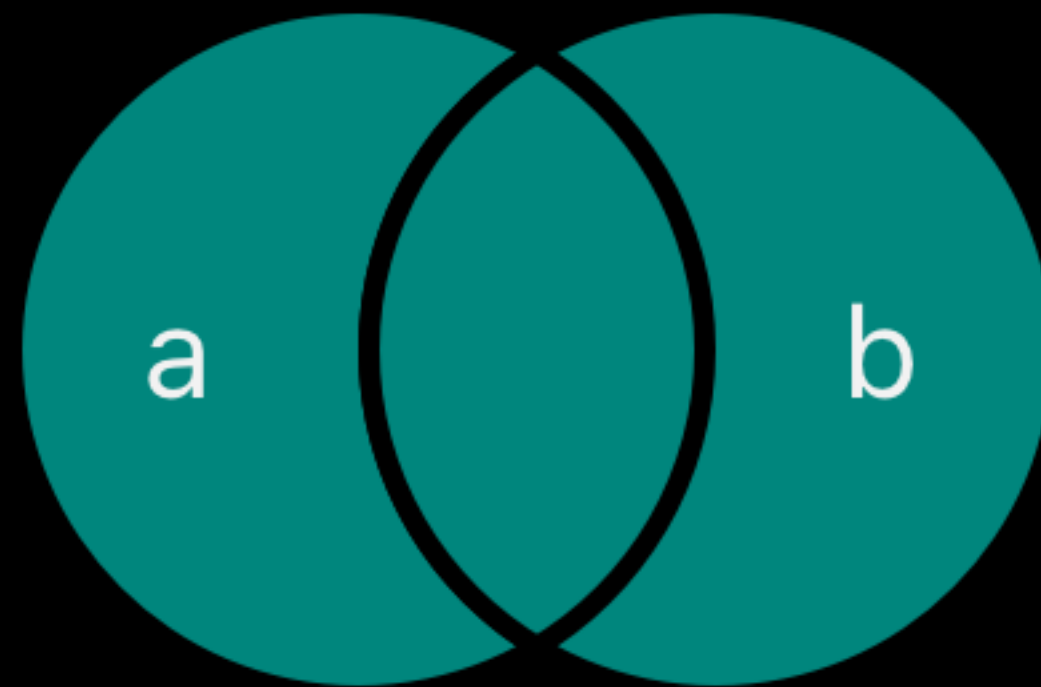
`a.intersection(b)`



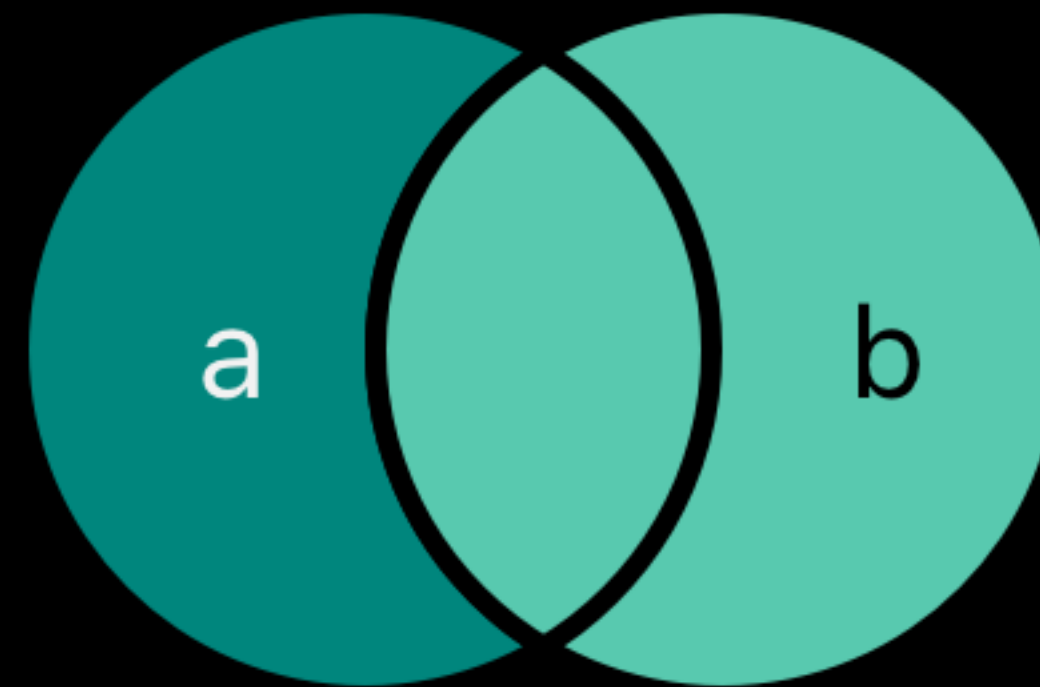
`a.symmetricDifference(b)`



`a.union(b)`



`a.subtracting(b)`



Fundamental Set Operations

- Use the `intersection(_:)` method to create a new set with only the values common to both sets.
- Use the `symmetricDifference(_:)` method to create a new set with values in either set, but not both.
- Use the `union(_:)` method to create a new set with all of the values in both sets.
- Use the `subtracting(_:)` method to create a new set with values not in the specified set.

Fundamental Set Operations

```
let oddDigits: Set = [1, 3, 5, 7, 9]
let evenDigits: Set = [0, 2, 4, 6, 8]
let singleDigitPrimeNumbers: Set = [2, 3, 5, 7]
```

```
oddDigits.union(evenDigits).sorted()
// [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
oddDigits.intersection(evenDigits).sorted()
// []
oddDigits.subtracting(singleDigitPrimeNumbers).sorted()
// [1, 9]
oddDigits.symmetricDifference(singleDigitPrimeNumbers).sorted()
// [1, 2, 9]
```


TUPLES

- A tuple is a group of different values and each value inside a tuple can be of different data types.
- Here's a simple Swift tuple example:

```
let productDetails = ("iPhone", 1099.99, true)
```

- Tuple groups different data types together in a comma separated list.

Tuple Syntax

- create a tuple by writing a comma separated list of values inside parentheses. Each value within the tuple can be of any data type, and you can access tuple elements by their index:

```
var person = ("John", 30)
```

```
print("Name: \(person.0)") // Accessing the first element
```

```
print("Age: \(person.1)")
```

Tuple Syntax

You can also give names to tuple elements when you define the tuple, making your code clearer:

```
var car = (make: "Toyota", model: "Corolla", year: 2021)
```

```
print("Car make: \(car.make)")
```

```
print("Car model: \(car.model)")
```


Accessing Tuple Elements by Index

- Each element in a tuple can be accessed using an index number, starting with zero.

```
let userInfo = ("Alice", 25)
```

```
let name = userInfo.0 // Access the first element
```

```
let age = userInfo.1 // Access the second element
```

```
print("Name: \(name), Age: \(age)")
```

Accessing Tuple Elements by Name

- Naming the elements in a tuple is optional, but it is a best practice when the tuple has more than two elements or when the context of each value isn't obvious.

```
let product = (id: 101, name: "Coffee", price: 2.99)
```

```
let productName = product.name
```

```
let productPrice = product.price
```

```
print("Product: \(productName), Price: $\(productPrice)")
```

Nested tuple

- Naming the elements in a tuple is optional, but it is a best practice when the tuple has more than two elements or when the context of each value isn't obvious.
- Tuples created as an element of another tuple are referred to as Nested Tuples.
- An example is shown below:

```
var alphaNum = ("x", 1, 2, ("y", 3, "z"))
```

- In the example above, we have a tuple ("y", 3, "z") as the third element of the created alphaNum tuple.

Unit 2—Lesson 5

Lab: Collections



Open and complete the exercises in Lab - Collections.playground

